

atividades módulo 4

PROJETO INTEGRADOR

Caderno de Estudos e Atividades do Estudante

Sprint 4 - Desenvolvimento de Sistemas | 17/08 a 21/08

Validação, execução, testes e implantação do banco PostgreSQL

Identificação da equipe

Preencha antes de iniciar as atividades da Sprint. Registre todos os integrantes e o papel assumido por cada estudante no desenvolvimento do projeto.

Nome do Projeto	Nome completo do estudante	Turma/Série	Papel no desenvolvimento do projeto
Educa inclui	Sabrina Natyellen Fiusa de Freitas	1ºB DS	Líder, Front-end
Educa inclui	Mariana Cuartas Carmona	1ºB DS	Back-end
Educa inclui	Kelly Caroline Michalak	1ºB DS	Banco de Dados
Educa inclui	Gabriella Dias Bueno	1ºB DS	Front-end
Educa inclui	Rafaela Pereira Vitorino	1ºB DS	Documentadora Técnica
Educa inclui	Lara Faccin Bonette	1ºA IA	Engenheiro de Prompts
Educa inclui	Kamilly Vitória Rosa Santo	1ºA IA	Cientista de Dados

Desafio da semana

O banco de dados que projetamos realmente funciona e está pronto para ser utilizado pelo sistema?

Como usar este caderno

Cada aula possui uma atividade prática para laboratório e uma atividade desplugada equivalente. Faça a modalidade indicada pelo professor. As duas produzem evidências para a mesma entrega e alimentam a aula seguinte.

Antes de começar

- Traga o Dicionário de Dados, o DER e o script SQL produzidos na Sprint anterior.
- Mantenha os arquivos do projeto organizados e versionados no GitHub.
- Não publique senhas, tokens, chaves ou connection strings com credenciais.
- Registre erros: nesta Sprint, um erro bem analisado é uma evidência de aprendizagem.
- Use IA para compreender, revisar e testar. Não copie uma solução sem verificar se ela respeita o projeto.

A jornada das 12 aulas

Aula	Ação	Resultado
1	Auditar	Dicionário × DER × SQL
2	Rastrear	Tela × ação × dado
3	Revisar	DDL PostgreSQL
4	Executar	Primeira criação do banco
5	Corrigir	Script reproduzível
6	Testar	CRUD
7	Desafiar	Integridade e testes negativos
8	Preparar	Supabase / banco remoto
9	Implantar	Schema no Supabase
10	Homologar	Local × remoto
11	Consolidar	Artefatos sincronizados
12	Entregar	GitHub + demonstração

Aula 1 - Auditoria: Dicionário × DER × SQL

O que vamos aprender	Coerência entre os três artefatos que descrevem o banco.
O que entregar	Tabela de inconsistências com evidências.

CONCEITO

O Dicionário descreve os dados; o DER representa entidades e relacionamentos; o DDL transforma a estrutura em comandos SQL. Os três precisam representar a mesma decisão de projeto.

Escolha o caminho da aula

NO LABORATÓRIO Abra os três artefatos lado a lado. Escolha uma tabela por vez e confira nome, campos, PK, FKs, obrigatoriedade e regras. Registre toda divergência antes de corrigir.	FORA DO LABORATÓRIO - DESPLUGADA Em folhas ou cartões, crie três áreas: DICIONÁRIO, DER e SQL. Escreva tabelas/campos em cartões e faça a correspondência física. Um cartão sem par é uma pendência a investigar.
---	---

Faça

1. Escolha uma entidade/tabela.
2. Localize-a no Dicionário.
3. Localize-a no DER.
4. Localize o CREATE TABLE correspondente no SQL.
5. Repita para os campos e relacionamentos.
6. Registre inconsistências sem apagá-las.

Registro da equipe

Item	Dicionário	DER	SQL / conclusão
IA COMO TUTORA Prompt: "Compare os artefatos que vou fornecer. Não redesenhe nem corrija ainda. Liste inconsistências objetivas entre Dicionário de Dados, DER e SQL e diga onde encontrou cada uma." Antes de aceitar qualquer apontamento, confirme no seu material.			

SAIBA MAIS - [PostgreSQL - Tutorial](#)

Introdução oficial ao PostgreSQL e à linguagem SQL.

Aula 2 - O banco sustenta o sistema?

O que vamos aprender	Relação entre interface, operações CRUD e persistência.
O que entregar	Mapa Tela → ação → tabela → campo.

CONCEITO

Persistir é guardar um dado para que possa ser recuperado depois. Uma tela pode ler ou escrever em várias tabelas. CRUD resume quatro operações: criar, ler, atualizar e excluir.

Escolha o caminho da aula

NO LABORATÓRIO Abra o mapa/protótipo de telas. Para cada campo e ação, identifique a tabela e coluna envolvidas. Marque C, R, U ou D.	FORA DO LABORATÓRIO - DESPLUGADA Crie cartões TELA, AÇÃO, TABELA e CAMPO. Monte trilhas físicas, por exemplo: "Cadastrar produto → salvar → produto → nome/preço". Campos sem destino viram pendências.
---	---

- Escolha uma tela.
- Liste os campos que aparecem nela.
- Classifique cada ação como C, R, U ou D.
- Aponte onde cada dado será armazenado ou consultado.
- Repita com pelo menos mais uma tela.

Tela	Ação	CRUD	Tabela	Campos
IA COMO REVISORA Prompt: "Analise esta tela e este Dicionário de Dados. Identifique campos ou ações que não possuem local claro de armazenamento/consulta. Não crie tabelas novas antes de explicar o problema."				

Aula 3 - Revisão técnica do DDL

O que vamos aprender	CREATE TABLE, tipos, PK, FK, NOT NULL, UNIQUE e compatibilidade com PostgreSQL.
O que entregar	schema_v1.sql revisado.

CONCEITOS-CHAVE

PRIMARY KEY identifica unicamente uma linha; FOREIGN KEY mantém o vínculo com outra tabela; NOT NULL exige valor; UNIQUE impede repetição. Restrições protegem a qualidade dos dados.

```
CREATE TABLE categoria (
  id_categoria INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  nome VARCHAR(100) NOT NULL UNIQUE
);
```

Escolha o caminho da aula

NO LABORATÓRIO Revise o arquivo SQL linha a linha. Marque CREATE TABLE, PK, FKs e constraints. Se o script veio de outro SGBD, procure sintaxes incompatíveis com PostgreSQL.	FORA DO LABORATÓRIO - DESPLUGADA Imprima/copie o DDL em papel. Use marcações: circule tabelas, sublinhe PK, faça setas nas FKs e destaque NOT NULL/UNIQUE. Faça uma "execução humana": leia de cima para baixo e verifique se tabelas referenciadas já existem.
---	---

12. Localize cada CREATE TABLE.
13. Identifique a PK de cada tabela.
14. Marque FKs e tabela referenciada.

15. Compare tipos e obrigatoriedade com o Dicionário.
16. Salve a versão candidata como schema_v1.sql.

IA COMO REVISORA DE SINTAXE

Prompt: “Este script deverá rodar em PostgreSQL. Revise a sintaxe. Para cada problema, explique o motivo e indique a menor correção possível. Não altere regras de negócio.”

SAIBA MAIS - [PostgreSQL - Constraints](#)

Documentação oficial sobre CHECK, NOT NULL, UNIQUE, PRIMARY KEY e FOREIGN KEY.

Aula 4 - Primeira execução do banco

O que vamos aprender

Execução, mensagem de erro, hipótese e evidência.

O que entregar

Registro de erros da primeira execução.

CONCEITO

Executar um script significa pedir ao SGBD que interprete os comandos. Uma mensagem de erro é uma pista: registre-a antes de tentar “consertar tudo”.

Escolha o caminho da aula

NO LABORATÓRIO

Em um PostgreSQL disponível, execute o schema_v1.sql em um ambiente vazio. Para cada falha, copie a mensagem, identifique o comando e formule uma hipótese.

FORA DO LABORATÓRIO - DESPLUGADA

Faça a “execução humana” do script. Um estudante lê cada comando; outro representa o banco e cria cartões de tabela. Quando aparecer uma FK, só aceite se o cartão da tabela referenciada já existir. Registre falhas lógicas/sintáticas suspeitas para testar depois.

Erro / suspeita	Comando	Nossa hipótese	Próxima tentativa
IA COMO EXPLICADORA Prompt: “Recebi esta mensagem de erro no PostgreSQL: [erro]. Explique em linguagem simples o que significa. Não reescreva todo o banco;			

diga primeiro onde devo procurar.”

SAIBA MAIS - [Supabase SQL Editor](#)

O SQL Editor também pode ser usado para executar consultas PostgreSQL diretamente no navegador.

Aula 5 - Corrigir até ser reproduzível

O que vamos aprender

Ciclo executar → compreender → corrigir → recriar → executar.

O que entregar

schema_v2.sql capaz de reconstruir o banco sem correções manuais ocultas.

CONCEITO

Reproduzibilidade significa que outra pessoa consegue recriar a mesma estrutura a partir do arquivo oficial. “Funcionou porque ajustamos manualmente depois” não é reproduzível.

Escolha o caminho da aula

NO LABORATÓRIO

Corrija uma falha por vez no arquivo fonte, recrie o ambiente e execute novamente. Repita até o script completo funcionar do início ao fim.

FORA DO LABORATÓRIO - DESPLUGADA

Use os cartões/tabelas da aula anterior. Para cada erro registrado, escreva em um novo cartão: causa provável → correção → como testar. Reordene fisicamente comandos quando a dependência exigir.

17. Escolha um erro.
18. Explique a causa com suas palavras.
19. Faça a menor correção possível.
20. Execute/revise novamente desde o início.
21. Registre se a hipótese foi confirmada.
22. Salve schema_v2.sql somente quando a sequência estiver consistente.

IA PARA CONFIRMAR APRENDIZAGEM

Prompt: “Explique por que esta alteração resolveu o erro. Quero confirmar se entendi e não apenas copiar a solução.”

Aula 6 - CRUD: provar que os dados podem ser usados

O que vamos aprender

INSERT, SELECT, UPDATE e DELETE.

O que entregar

testes_crud.sql + resultados esperados/obtidos.

CONCEITO

CRUD representa Create, Read, Update e Delete. Em SQL, normalmente usamos INSERT, SELECT, UPDATE e DELETE. Antes de executar um teste, declare o resultado esperado.

```
INSERT INTO categoria (nome) VALUES ('Bebidas');
SELECT * FROM categoria;
UPDATE categoria SET nome='Bebidas sem álcool' WHERE id_categoria=1;
DELETE FROM categoria WHERE id_categoria=1;
```

Escolha o caminho da aula

NO LABORATÓRIO

Escolha uma tabela simples e uma tabela relacionada. Crie dados fictícios e execute as quatro operações. Compare resultado esperado e obtido.

FORA DO LABORATÓRIO - DESPLUGADA

Simule o banco com fichas: cada ficha é uma linha. INSERT adiciona ficha; SELECT localiza; UPDATE altera; DELETE remove. Um colega faz o papel das constraints e pode impedir operações inválidas.

Operação	Comando/simulação	Esperado	Obtido
INSERT			
SELECT			
UPDATE			
DELETE			
IA PARA DADOS DE TESTE Prompt: "Crie dados fictícios coerentes para testar estas tabelas. Não altere minha estrutura. Gere apenas exemplos de INSERT e explique a ordem necessária por causa das FKs."			

SAIBA MAIS - [PostgreSQL - Inserindo dados](#)

Referência oficial para INSERT e manipulação de dados.

Aula 7 - Testes negativos e integridade

O que vamos aprender	Testar também aquilo que o banco deve rejeitar.
O que entregar	Matriz de testes de integridade.

CONCEITO

Um teste positivo verifica o que deve funcionar. Um teste negativo tenta violar uma regra de propósito. Se um email é

UNIQUE, repetir o mesmo email deve falhar. Se uma FK exige um registro pai, apontar para um ID inexistente deve falhar.

Escolha o caminho da aula

NO LABORATÓRIO

Crie testes que violem NOT NULL, UNIQUE, PK e FK existentes no seu projeto. Antes de rodar, escreva o resultado esperado.

FORA DO LABORATÓRIO - DESPLUGADA

Monte uma “portaria do banco”: cartões representam registros e estudantes representam constraints. Cada cartão de entrada é aceito ou recusado, mas a equipe deve citar a regra que justifica a decisão.

Teste	Regra	Esperado	Obtido	Passou?
IA COMO GERADORA DE CENÁRIOS Prompt: “Para estas tabelas PostgreSQL, proponha testes negativos para verificar as constraints que realmente existem no meu script. Não invente regras novas.”				

SAIBA MAIS - [PostgreSQL - Constraints](#)

Use a documentação para confirmar o comportamento das restrições.

Aula 8 - Do banco local ao banco remoto

O que vamos aprender	Servidor, credenciais, SQL Editor, Table Editor e segurança.
O que entregar	Plano de implantação + projeto Supabase preparado (quando houver laboratório).

CONCEITO

No Supabase, cada projeto recebe um banco PostgreSQL completo. O Dashboard possui SQL Editor para executar

SQL e Table Editor para visualizar/editar tabelas e linhas. O banco remoto poderá ser acessado pelo backend em outra etapa.

Escolha o caminho da aula

NO LABORATÓRIO Acesse o Supabase, crie/abra o projeto da equipe, reconheça Database, SQL Editor, Table Editor e Connect. Guarde a senha com segurança e NÃO a coloque no GitHub.	FORA DO LABORATÓRIO - DESPLUGADA Desenhe a arquitetura em uma folha: COMPUTADOR/APP → REDE → SERVIDOR → POSTGRESQL. Crie cartões "host", "porta", "usuário", "senha" e "banco". Classifique quais informações podem aparecer em documentação e quais são segredo.
--	---

23. Identifique onde o banco ficará hospedado.
24. Localize SQL Editor e Table Editor.
25. Identifique onde ficam as opções de conexão (Connect).
26. Defina quem guardará a credencial da equipe.
27. Revise o repositório para garantir que não há senhas.

SEGURANÇA

Nunca coloque senha, token ou connection string completa com segredo em repositório público, print compartilhado ou prompt de IA.

SAIBA MAIS - [Supabase - Database](#)

Visão geral oficial: PostgreSQL, Table Editor, SQL Editor e formas de acesso.

Aula 9 - Implantação no Supabase

O que vamos aprender	Implantar é criar no ambiente remoto a estrutura já validada.
O que entregar	Banco remoto criado a partir do schema.sql.

CONCEITO

O arquivo schema.sql continua sendo a fonte oficial. Se houver erro estrutural, corrija o arquivo e depois reaplique; não deixe uma "versão secreta" que só existe no Dashboard.

Escolha o caminho da aula

NO LABORATÓRIO No Supabase: abra SQL Editor → New Query → cole o schema.sql validado → Run. Depois confira as tabelas no Table Editor. Registre qualquer erro.	FORA DO LABORATÓRIO - DESPLUGADA Faça uma implantação simulada: uma folha representa o servidor remoto vazio. Execute o schema em cartões, criando tabelas na ordem. Um "auditor remoto" só aceita o que está no arquivo oficial.
--	---

28. Confirme que está usando a versão final do schema.
29. Abra o SQL Editor.
30. Crie uma nova consulta.

31. Execute o schema.
32. Confira tabelas e relacionamentos.
33. Registre evidências da implantação sem expor credenciais.

PERGUNTA DE CONTROLE

Se o banco remoto fosse apagado agora, sua equipe conseguiria reconstruí-lo usando somente os arquivos versionados? A resposta precisa ser SIM.

SAIBA MAIS - [Supabase - SQL Editor](#)

Interface do Supabase para escrever e executar SQL no navegador.

Aula 10 - Homologação: local × remoto

O que vamos aprender

Repetir os testes no ambiente implantado e comparar comportamentos.

O que entregar

Relatório de homologação local × remoto.

CONCEITO

Homologar é verificar se o ambiente implantado apresenta o comportamento esperado. Criar as tabelas não basta: os testes precisam confirmar CRUD e integridade.

Escolha o caminho da aula

NO LABORATÓRIO

Reexecute testes_crud.sql e os testes negativos no Supabase. Compare os resultados com o ambiente anterior e investigue diferenças.

FORA DO LABORATÓRIO - DESPLUGADA

Use duas áreas na sala: LOCAL e REMOTO. Reaplique os mesmos cartões de teste nos dois “ambientes”. Qualquer resultado diferente precisa gerar uma hipótese de investigação.

Teste	Local	Remoto	Conclusão
IA PARA INVESTIGAÇÃO Prompt: “Este teste teve resultado diferente entre dois ambientes PostgreSQL. Ajude-me a criar hipóteses de			

investigação sem mudar a regra de negócio nem expor credenciais.”

Aula 11 - Consolidar: tudo precisa dizer a mesma coisa

O que vamos aprender	Sincronizar Dicionário, DER, schema e banco implantado.
O que entregar	Versões finais dos artefatos + checklist de auditoria.

CONCEITO

Documentação desatualizada também é defeito. Ao final, a estrutura descrita no Dicionário e DER deve corresponder ao SQL que reconstrói o banco implantado.

Escolha o caminho da aula

NO LABORATÓRIO Compare Dicionário, DER, schema.sql e estrutura no Supabase. Faça correções finais nos arquivos oficiais e uma auditoria cruzada com outra equipe.	FORA DO LABORATÓRIO - DESPLUGADA Monte quatro estações: DICIONÁRIO, DER, SQL e BANCO IMPLANTADO (representado por desenho/lista). Grupos rodam pelas estações e marcam divergências em post-its sem editar o trabalho da outra equipe.
---	--

Checklist

- ☐ Toda tabela aparece nos quatro lugares.
- ☐ Nomes de campos estão coerentes.
- ☐ Tipos são compatíveis.
- ☐ PKs e FKs correspondem.
- ☐ Obrigatoriedade/restrições correspondem.
- ☐ Toda alteração feita durante os testes voltou para a documentação.

IA COMO AUDITORA

Prompt: “Faça uma auditoria de consistência entre estes artefatos. Não redesenhe a solução. Liste divergências objetivas em nomes, tipos, obrigatoriedade, PK, FK e regras.”

Aula 12 - Versionar, entregar e demonstrar

O que vamos aprender	Entrega profissional combina código, documentação, testes, segurança e explicação.
O que entregar	Repositório final organizado + demonstração + pendências para a próxima Sprint.

CONCEITO

O projeto não termina quando “funciona no meu computador”. A equipe deve permitir que outra pessoa compreenda, reconstrua e teste o banco.

Escolha o caminho da aula

NO LABORATÓRIO

Organize /docs e /database, revise segredos, faça commit e demonstre no Supabase: uma tabela, um CRUD e um teste negativo.

FORA DO LABORATÓRIO - DESPLUGADA

Realize uma banca técnica com cartões-surpresa: explique uma entidade, sua PK, um relacionamento, simule um INSERT válido e um inválido. Use os artefatos impressos como evidência.

```
/docs
dicionario_dados
DER
relatorio_validacao
/database
schema.sql
testes_crud.sql
testes_integridade.sql
```

Atividade transversal - Publicação da Sprint no Instagram do projeto

Toda semana de Projeto Integrador, a equipe deverá publicar uma evidência do desenvolvimento realizado no Instagram do próprio projeto. A postagem faz parte da entrega da Sprint.

Nesta semana, a publicação deve contar a transformação do banco projetado em um banco executável, testado e implantado.

1. Escolha uma evidência real da Sprint: DER, trecho do schema.sql, execução de teste, tabela criada no Supabase ou outro registro do trabalho da equipe.
2. Explique, em linguagem acessível, o que a equipe fez nesta semana.
3. Conte pelo menos um problema encontrado durante execução, teste ou implantação e como a equipe investigou/corrigiu.
4. Mostre o avanço obtido: o que o projeto passou a conseguir fazer depois desta Sprint?
5. Revise imagem e texto antes de publicar. Não exponha senha, token, chave, connection string, endereço privado ou qualquer credencial.
6. Publique no Instagram do projeto e registre a evidência/link da postagem no material de entrega da equipe.

Uso da IA na publicação

A IA pode ajudar a transformar o registro técnico da Sprint em uma legenda compreensível. Exemplo de prompt: “Com base neste resumo do que realmente fizemos, ajude nossa equipe a escrever uma legenda curta para o Instagram do projeto. Explique o avanço técnico em linguagem acessível, sem inventar resultados e sem incluir credenciais.”

Importante: a equipe deve conferir se a publicação representa aquilo que foi realmente desenvolvido. IA ajuda a comunicar; não cria uma entrega que não aconteceu.

Registro da publicação	Preenchimento da equipe
Evidência escolhida	
Problema/avanço que será contado	
Legenda revisada pela equipe?	
Link ou identificação da publicação	

Checklist de entrega

- ☐ Dicionário, DER e schema.sql coerentes.
- ☐ Schema reproduzível.
- ☐ Testes CRUD registrados.
- ☐ Testes negativos registrados.
- ☐ Banco remoto homologado.
- ☐ Nenhum segredo versionado.
- ☐ Arquivos organizados no GitHub.
- ☐ Equipe consegue explicar uma entidade, um relacionamento, um CRUD e um teste negativo.
- ☐ Publicação semanal no Instagram do projeto realizada e registrada, sem exposição de credenciais.

IA COMO BANCA

Prompt: “Atue como banca técnica. Faça uma pergunta por vez para verificar se entendemos nosso banco, testes e implantação. Espere nossa tentativa antes de explicar a resposta.”

Guia rápido de SQL para esta Sprint

CREATE TABLE

Cria a estrutura de uma tabela. O DDL deve refletir as decisões do Dicionário e do DER.

```
CREATE TABLE produto (  
  id_produto INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  nome VARCHAR(120) NOT NULL,  
  preco NUMERIC(10,2) NOT NULL  
);
```

INSERT / SELECT / UPDATE / DELETE

São os comandos básicos usados nesta Sprint para verificar CRUD. Sempre use dados fictícios de teste e confira o resultado.

SAIBA MAIS - [PostgreSQL - Tutorial oficial](#)

Percurso introdutório oficial sobre bancos relacionais e SQL.

SAIBA MAIS - [PostgreSQL - UPDATE](#)

Exemplos oficiais de atualização de linhas.

Como usar IA sem terceirizar sua aprendizagem

34. Antes do prompt, escreva sua hipótese.
35. Peça explicação antes de pedir código.
36. Compare a resposta com Dicionário, DER e regras do projeto.
37. Execute ou simule a proposta.
38. Registre a evidência que confirmou ou rejeitou a sugestão.

PROMPT-BASE

“Somos estudantes desenvolvendo um projeto autoral com PostgreSQL. Atue como tutor. Primeiro explique o conceito necessário, depois ajude a localizar a causa, proponha passos de investigação e só então uma possível correção. Termine dizendo como testar se funcionou.”

Papéis da equipe

Papel	Responsabilidade
Operador	Executa comandos quando houver computador.
Auditor	Compara Dicionário, DER, SQL e resultados.
Analista de erro	Registra mensagens, hipóteses e evidências.
Testador	Cria cenários positivos e negativos.
Documentador	Atualiza relatórios e checklists.
Consultor de IA	Formula prompts e valida respostas.
Guardião de segurança	Confere que segredos não sejam expostos.
ROTACIONE Troque os papéis ao longo da Sprint. O estudante que já domina mais tecnologia não deve ser o único a operar o computador.	

Saiba mais - referências oficiais

- [PostgreSQL Tutorial](#)
- [PostgreSQL Constraints](#)
- [PostgreSQL INSERT](#)
- [Supabase Database](#)
- [Supabase SQL Editor](#)

aula 1

Aula 1 - Auditoria: Dicionário × DER × SQL

O que vamos aprender	Coerência entre os três artefatos que descrevem o banco.
O que entregar	Tabela de inconsistências com evidências.

CONCEITO

O Dicionário descreve os dados; o DER representa entidades e relacionamentos; o DDL transforma a estrutura em comandos SQL. Os três precisam representar a mesma decisão de projeto.

Escolha o caminho da aula

NO LABORATÓRIO Abra os três artefatos lado a lado. Escolha uma tabela por vez e confira nome, campos, PK, FKs, obrigatoriedade e regras. Registre toda divergência antes de corrigir.	FORA DO LABORATÓRIO - DESPLUGADA Em folhas ou cartões, crie três áreas: DICIONÁRIO, DER e SQL. Escreva tabelas/campos em cartões e faça a correspondência física. Um cartão sem par é uma pendência a investigar.
---	---

Faça

1. Escolha uma entidade/tabela.
2. Localize-a no Dicionário.
3. Localize-a no DER.
4. Localize o CREATE TABLE correspondente no SQL.
5. Repita para os campos e relacionamentos.
6. Registre inconsistências sem apagá-las.

Registro da equipe

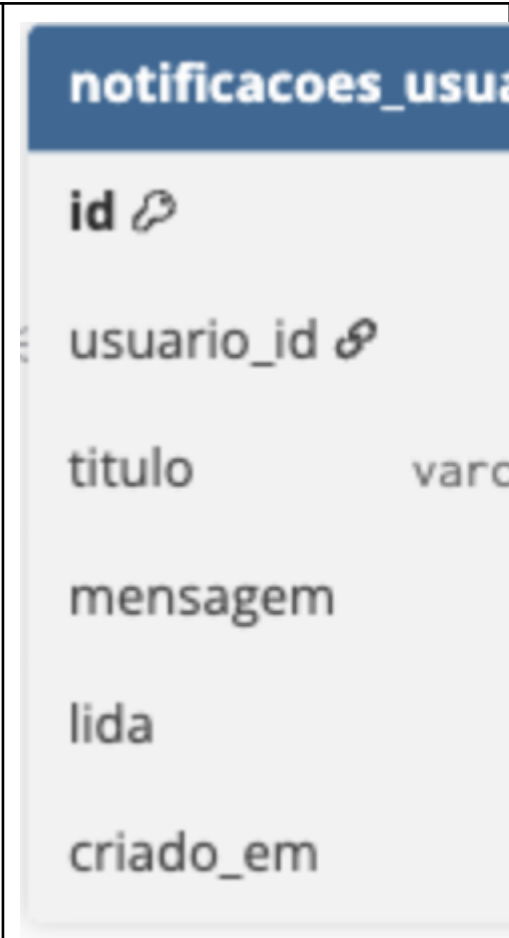
Item	Dicionário	DER	SQL / conclusão
IA COMO TUTORA Prompt: "Compare os artefatos que vou fornecer. Não redesenhe nem corrija ainda. Liste inconsistências objetivas entre Dicionário de Dados, DER e SQL e diga onde encontrou cada uma." Antes de aceitar qualquer apontamento, confirme no seu material.			

SAIBA MAIS - [PostgreSQL - Tutorial](#)
Introdução oficial ao PostgreSQL e à linguagem SQL.

	Dicionário de dados	Diagrama entidade-relacionamento	DDL
			<pre>CREATE TABLE public.usuarios (id SERIAL PRIMARY KEY, nome VARCHAR(150) NOT NULL, email VARCHAR(150) UNIQUE NOT NULL, senha_hash VARCHAR(255) NOT NULL, tipo VARCHAR(50) NOT NULL, data_nascimento DATE, ativo BOOLEAN DEFAULT TRUE, criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP, atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP);</pre>

		turmas id nome ano_letivo descricao criado_em	<pre>CREATE TABLE public.turmas (id SERIAL PRIMARY KEY, nome VARCHAR(100) NOT NULL, ano_letivo INT NOT NULL, descricao TEXT, criado_em TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP);</pre>
		matriculas_turma aluno_id turma_id data_matricula status	<pre>CREATE TABLE public.matriculas_turma (aluno_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE, turma_id INT NOT NULL REFERENCES public.turmas(id) ON DELETE CASCADE, data_matricula TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP, status VARCHAR(50) DEFAULT 'ativo', PRIMARY KEY (aluno_id, turma_id));</pre>

lor da o
io
aviso
da o
de o
do



```
CREATE TABLE
public.notificacoes_usuario (
  id SERIAL PRIMARY KEY,
  usuario_id INT NOT NULL
REFERENCES
public.usuarios(id) ON DELETE
CASCADE,
  titulo VARCHAR(150) NOT
NULL,
  mensagem TEXT NOT NUL
  lida BOOLEAN DEFAULT
FALSE,
  criado_em TIMESTAMP
DEFAULT
CURRENT_TIMESTAMP
);
```

ção de
etário
na aste
anho
n s
a leitor
e
fia exia
ra

perfis_acessibilid
id 🔑
usuario_id 🔗 📱
alto_contraste
tamanho_fonte
ativar_vlibras
leitor_tela_nativo
reduzir_animacoes
fonte_dislexia
criado_em

```
CREATE TABLE
public.perfis_acessibilidade (
  id SERIAL PRIMARY KEY,
  usuario_id INT UNIQUE NOT
NULL REFERENCES
public.usuarios(id) ON DELETE
CASCADE,
  alto_contraste BOOLEAN
DEFAULT FALSE,
  tamanho_fonte VARCHAR(2
DEFAULT 'padrao',
  ativar_vlibras BOOLEAN
DEFAULT FALSE,
  leitor_tela_nativo BOOLEAN
DEFAULT FALSE,
  reduzir_animacoes BOOLEA
DEFAULT FALSE,
  fonte_dislexia BOOLEAN
DEFAULT FALSE,
  criado_em TIMESTAMP
DEFAULT
CURRENT_TIMESTAMP
);
```

ssidad
ex:
icativa
cional

necessidades_esp

id

nome

descricao

codigo_cid

```
CREATE TABLE
public.necessidades_especiais
  id SERIAL PRIMARY KEY,
  nome VARCHAR(100) NOT
NULL UNIQUE,
  descricao TEXT,
  codigo_cid VARCHAR(20)
);
```

ack (mo)
onde ação
rio ncia
la
lo

feedback_acessib

id

usuario_id

pagina_url

comentario

nota

criado_em

```
CREATE TABLE
public.feedback_acessibilidade
  id SERIAL PRIMARY KEY,
  usuario_id INT REFERENC
public.usuarios(id) ON DELET
SET NULL,
  pagina_url VARCHAR(255)
NOT NULL,
  comentario TEXT NOT NUL
  nota INT CHECK (nota
BETWEEN 1 AND 5),
  criado_em TIMESTAMP
DEFAULT
CURRENT_TIMESTAMP
);
```

		usuario_necessid usuario_id necessidade_id criado_em	<pre>CREATE TABLE public.usuario_necessidades_es ais (usuario_id INT NOT NULL REFERENCES public.usuarios(ON DELETE CASCADE, necessidade_id INT NOT NU REFERENCES public.necessidades_especiais(ON DELETE CASCADE, criado_em TIMESTAMP DEFAULT CURRENT_TIMESTA PRIMARY KEY (usuario_id, necesidade_id));</pre>
		tecnologias_assis id usuario_id nome_tecnologia versao atualizado_em	<pre>CREATE TABLE public.tecnologias_assistivas_us o (id SERIAL PRIMARY KEY, usuario_id INT NOT NULL REFERENCES public.usuarios(ON DELETE CASCADE, nome_tecnologia VARCHAR(NOT NULL, versao VARCHAR(20), atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTA);</pre>

a atéria (ex: disciplina o na		materias id 🔑 nomevar descricao ordem	CREATE TABLE public.materia (id SERIAL PRIMARY KEY, nome VARCHAR(150) NOT NULL, descricao TEXT, ordem INT DEFAULT 1);

Titulo de origem unidade do módulo a do módulo		modulos id materia_id titulo descricao ordem	CREATE TABLE public.modulos (id SERIAL PRIMARY KEY, materia_id INT NOT NULL REFERENCES public.materias(id) ON DELETE CASCADE, titulo VARCHAR(150) NOT NULL, descricao TEXT, ordem INT DEFAULT 1);
o dente ção a sequência édio para		licoes id modulo_id titulo ordem duracao_estimada	CREATE TABLE public.licoes (id SERIAL PRIMARY KEY, modulo_id INT NOT NULL REFERENCES public.modulos(id) ON DELETE CASCADE, titulo VARCHAR(150) NOT NULL, ordem INT DEFAULT 1, duracao_estimada_minutos INT DEFAULT 15);

teúdo lor do texto, vídeo, (F) arkdown quivo de io textual sibilidade		conteudos id 🔑 titulo tipo corpo_texto url_midia transcricao_audio	<pre>CREATE TABLE public.conteudos (id SERIAL PRIMARY KEY, titulo VARCHAR(150) NOT NULL, tipo VARCHAR(50) NOT NULL, corpo_texto TEXT, url_midia VARCHAR(255), transcricao_audio TEXT);</pre>
exibe o atrelado conteúdo		conteudos_licao licao_id 🔑 🔗 conteudo_id 🔑 🔗 ordem	<pre>CREATE TABLE public.conteudos_licao (licao_id INT NOT NULL REFERENCES public.licoes(id) ON DELETE CASCADE, conteudo_id INT NOT NULL REFERENCES public.conteudos(id) ON DELETE CASCADE, ordem INT DEFAULT 1, PRIMARY KEY (licao_id, conteudo_id));</pre>

alternativa associada da opção ta da opção		alternativas id questao_id texto_alternativa is_correta	<pre>CREATE TABLE public.alternativas (id SERIAL PRIMARY KEY, questao_id INT NOT NULL REFERENCES public.questoes(id) ON DELETE CASCADE, texto_alternativa TEXT NOT NULL, is_correta BOOLEAN DEFAULT FALSE);</pre>
missão respondeu respondida a marcada caso de berta do acerto da		respostas_usuario id usuario_id questao_id alternativa_id resposta_dissertat acertou respondido_em	<pre>CREATE TABLE public.respostas_usuario (id SERIAL PRIMARY KEY, usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE, questao_id INT NOT NULL REFERENCES public.questoes(id) ON DELETE CASCADE, alternativa_id INT REFERENCES public.alternativas(id), resposta_dissertativa TEXT, acertou BOOLEAN, respondido_em TIMESTAM DEFAULT CURRENT_TIMESTAMP);</pre>

acompanhado pendida finalização termino do		progresso_licoes usuario_id licao_id concluido data_conclusao	CREATE TABLE public.progresso_licoes (usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE, licao_id INT NOT NULL REFERENCES public.licoes(id) ON DELETE CASCADE, concluido BOOLEAN DEFAULT FALSE, data_conclusao TIMESTAM PRIMARY KEY (usuario_id, licao_id));
istro de estudo e entrada e saída calculada da		sessoes_estudo id usuario_id inicio fim duracao_segundos	CREATE TABLE public.sesoes_estudo (id SERIAL PRIMARY KEY, usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE, inicio TIMESTAMP DEFAULT CURRENT_TIMESTAMP, fim TIMESTAMP, duracao_segundos INT);

conquista medalha para r icone ativo o XP		conquistas id 🔑 nome descricao icone_url pontos	<pre>CREATE TABLE public.conquistas (id SERIAL PRIMARY KEY, nome VARCHAR(100) NOT NULL, descricao TEXT NOT NULL, icone_url VARCHAR(255), pontos INT DEFAULT 10);</pre>
entor da alcançada anho		conquistas_usuario usuario_id 🔑 🔗 conquista_id 🔑 🔗 desbloqueado_em	<pre>CREATE TABLE public.conquistas_usuario (usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE, conquista_id INT NOT NULL REFERENCES public.conquistas(id) ON DELETE CASCADE, desbloqueado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP, PRIMARY KEY (usuario_id, conquista_id));</pre>

recompensa geradora do item (ma, título) lor do item		recompensas_cor id 🔑 conquista_id 🔗 tipo_recompensa valor_recompensa	<pre>CREATE TABLE public.recompensas_conquista id SERIAL PRIMARY KEY, conquista_id INT NOT NULL REFERENCES public.conquistas(id) ON DELETE CASCADE, tipo_recompensa VARCHAR(50) NOT NULL, valor_recompensa VARCHAR(255) NOT NULL);</pre>
onitorado o a no naturidade a diária de studos ltimo ntabilizado		progresso_gamifi usuario_id 🔑 🔗 pontos_xp nivel dias_ofensiva ultima_atividade	<pre>CREATE TABLE public.progresso_gamificacao usuario_id INT PRIMARY KEY REFERENCES public.usuarios(id) ON DELETE CASCADE, pontos_xp INT DEFAULT 0, nivel INT DEFAULT 1, dias_ofensiva INT DEFAULT ultima_atividade DATE DEFAULT CURRENT_DATE);</pre>

alho/deck o deck conjunto de o assunto riação		decks_flashcards id usuario_id titulo descricao criado_em	CREATE TABLE public.decks_flashcards_usuar (id SERIAL PRIMARY KEY, usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE, titulo VARCHAR(150) NOT NULL, descricao TEXT, criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
ção de ção ual faz da frente (studo) do verso ento de paçada		cards_memorizac id deck_id frente verso proxima_revisao	CREATE TABLE public.cards_memorizacao_us rio (id SERIAL PRIMARY KEY, deck_id INT NOT NULL REFERENCES public.decks_flashcards_usuar (id) ON DELETE CASCADE, frente TEXT NOT NULL, verso TEXT NOT NULL, proxima_revisao DATE DEFAULT CURRENT_DATE);

de IA ndido pela ação erida (ex: ódulo X) enico da o algoritmo da geração		logs_aprendizado id 🔑 usuario_id 🔗 acao_recomendada justificativa_ia gerado_em	<pre>CREATE TABLE public.logs_aprendizado_ia (id SERIAL PRIMARY KEY, usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE, acao_recomendada VARCHAR(150) NOT NULL, justificativa_ia TEXT, gerado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP);</pre>
mento de nado nada para ncia da revisão ra refazer o		mapeamento_rev id 🔑 usuario_id 🔗 licao_id 🔗 prioridade data_sugerida_rev	<pre>CREATE TABLE public.mapeamento_revisao_ia (id SERIAL PRIMARY KEY, usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE, licao_id INT NOT NULL REFERENCES public.licoes(id) ON DELETE CASCADE, prioridade INT DEFAULT 1, data_sugerida_revisao DAT NOT NULL);</pre>

o longo me de
realizou
hado (traste')
nto tivou',
lado com o
o da

logs_acessibilidade
id 🔑
usuario_id 🔗
recurso_acessibilidade
acao_usuario
tempo_uso_segundos
criado_em

```
CREATE TABLE
public.logs_acessibilidade_ia (
    id BIGSERIAL PRIMARY KEY,
    usuario_id INT NOT NULL
REFERENCES public.usuarios(
ON DELETE CASCADE,
recurso_acessibilidade_utiliza
VARCHAR(100) NOT NULL,
acao_usuario VARCHAR(50)
NOT NULL,
tempo_uso_segundos INT,
criado_em TIMESTAMP
DEFAULT CURRENT_TIMESTAMP
);
```

co do fórum ue publicou ítulo da o alhado da discussão ra da o		forum_topicos id 🔑 autor_id 🔗 titulo varc conteudo criado_em	CREATE TABLE public.forum_topicos (id SERIAL PRIMARY KEY, autor_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE, titulo VARCHAR(200) NOT NULL, conteudo TEXT NOT NULL, criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
posta i onde foi o utor da o/resposta da		forum_respostas id 🔑 topico_id 🔗 autor_id 🔗 resposta criado_em	CREATE TABLE public.forum_respostas (id SERIAL PRIMARY KEY, topico_id INT NOT NULL REFERENCES public.forum_topicos(id) ON DELETE CASCADE, autor_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE, resposta TEXT NOT NULL, criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP);

aula 2

Aula 2 - O banco sustenta o sistema?

O que vamos aprender	Relação entre interface, operações CRUD e persistência.
O que entregar	Mapa Tela → ação → tabela → campo.

CONCEITO

Persistir é guardar um dado para que possa ser recuperado depois. Uma tela pode ler ou escrever em várias tabelas. CRUD resume quatro operações: criar, ler, atualizar e excluir.

Escolha o caminho da aula

NO LABORATÓRIO Abra o mapa/protótipo de telas. Para cada campo e ação, identifique a tabela e coluna envolvidas. Marque C, R, U ou D.	FORA DO LABORATÓRIO - DESPLUGADA Crie cartões TELA, AÇÃO, TABELA e CAMPO. Monte trilhas físicas, por exemplo: "Cadastrar produto → salvar → produto → nome/preço". Campos sem destino viram pendências.
---	---

1. Escolha uma tela.
2. Liste os campos que aparecem nela.
3. Classifique cada ação como C, R, U ou D.
4. Aponte onde cada dado será armazenado ou consultado.
5. Repita com pelo menos mais uma tela.

Tela	Ação	CRUD	Tabela	Campos

IA COMO REVISORA

Prompt: "Análise esta tela e este Dicionário de Dados. Identifique campos ou ações que não possuem local claro de armazenamento/consulta. Não crie tabelas novas antes de explicar o problema."

CRUD	Tabela do Banco	Campo(s) Impactado(s)
R	Aluno	email_matricula, senha
U	Configuracao_Acessibilidade	tamanho_fonte, contraste_cores, sintese_voz, velocidade
R	Aluno	nome_completo, data_nascimento, cpf, matricula, email
U	Aluno	senha
R	Atividade	titulo, descricao_instrucoes, anexos_url, data_hora_inicio, data_hora_termino
C	Resposta_Atividade	cpf_aluno, id_atividade, opcao_resposta, data_hora_inicio, data_hora_termino
R	Resposta_Atividade / Aluno	Campo Calculado Back-end: nota_obtida, historico_respostas
R	Professor	email_professor, senha
U	Professor	foto_perfil_url, telefone, tema_exibicao
R	Turma	codigo_turma, nome_turma, disciplina, ano_letivo
U	Turma	mural_recados, cronograma_conteudos
C	Atividade	codigo_turma, titulo, descricao_instrucoes, tipo, data_hora_inicio, data_hora_termino
D	Atividade	id_atividade
U	Resposta_Atividade	nota_obtida, observacoes_pedagogicas
R	Secretaria_Usuario	email_usuario, senha
U	Secretaria_Usuario / Configuracao_Acessibilidade	ano_letivo_vigente, dados_instituicao, parametros_acessibilidade
C	Turma	codigo_turma, nome_turma, disciplina, serie_ano, data_hora_inicio, data_hora_termino
U	Turma	email_professor, status_turma
D	Turma	codigo_turma
R	Atividade	titulo, descricao_instrucoes, anexos_url, data_hora_inicio, data_hora_termino
C	Resposta_Atividade	cpf_aluno, id_atividade, opcao_resposta, data_hora_inicio, data_hora_termino
C	Atividade	codigo_turma, titulo, descricao_instrucoes, tipo, data_hora_inicio, data_hora_termino
D	Atividade	id_atividade

NOTE: Aula 2 foi feita por Mariana, qualquer erro/dúvida perguntar a ela;
 Possíveis erros dessa planilha: na coluna da tabela do banco, eu acredito que todas essas telas tem haver somente com uma única tabela: a tabela usuarios

aula 3

Aula 3 - Revisão técnica do DDL

O que vamos aprender	CREATE TABLE, tipos, PK, FK, NOT NULL, UNIQUE e compatibilidade com PostgreSQL.
O que entregar	schema_v1.sql revisado.

CONCEITOS-CHAVE

PRIMARY KEY identifica unicamente uma linha; FOREIGN KEY mantém o vínculo com outra tabela; NOT NULL exige valor; UNIQUE impede repetição. Restrições protegem a qualidade dos dados.

```
CREATE TABLE categoria (  
  id_categoria INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL UNIQUE  
);
```

Escolha o caminho da aula

NO LABORATÓRIO Revise o arquivo SQL linha a linha. Marque CREATE TABLE, PK, FKs e constraints. Se o script veio de outro SGBD, procure sintaxes incompatíveis com PostgreSQL.	FORA DO LABORATÓRIO - DESPLUGADA Imprima/copie o DDL em papel. Use marcações: circule tabelas, sublinhe PK, faça setas nas FKs e destaque NOT NULL/UNIQUE. Faça uma “execução humana”: leia de cima para baixo e verifique se tabelas referenciadas já existem.
---	---

1. Localize cada CREATE TABLE.
2. Identifique a PK de cada tabela.
3. Marque FKs e tabela referenciada.
4. Compare tipos e obrigatoriedade com o Dicionário.
5. Salve a versão candidata como schema_v1.sql.

IA COMO REVISORA DE SINTAXE

Prompt: “Este script deverá rodar em PostgreSQL. Revise a sintaxe. Para cada problema, explique o motivo e indique a menor correção possível. Não altere regras de negócio.”

1. Valide a ordem de criação das tabelas (Execução Humana):

tabela **usuario_necessidades_especiais** veio antes da tabela **necessidades_especiais**, então deverá ser arrumado a ordem, colocando ela depois da tabela **necessidades_especiais**.

2. Revise a sintaxe para PostgreSQL

- Confirme se o identificador automático usa a sintaxe padrão do PostgreSQL: **Sim, o código em sql está com a sintaxe padrão do Postgres**
- Verifique se os tipos de dados são suportados no PostgreSQL: **Sim, todos os tipos de dados usados são suportados no Postgres**(note: possível chance do tipo de dado BIGSERIAL não ser aceito)

3. Verifique as Restrições (Constraints)

- **PRIMARY KEY (PK):** Toda tabela possui uma chave primária claramente definida? **Sim**
- **FOREIGN KEY (FK):** As chaves estrangeiras apontam corretamente para a tabela e o campo correspondentes? **Sim**
- **NOT NULL:** Todos os campos definidos como obrigatórios no Dicionário receberam a instrução **NOT NULL**? **Receberam**
- **UNIQUE:** Os campos que não podem ter valores repetidos (como **cpf**, **cnpj** ou **email**) contêm a restrição **UNIQUE**? **Sim**

Legenda de Cores

Categoria	Elemento	Cor / Exemplo
Chaves & Identificadores	Tabela Principal (Entidade)	NOME_DA_TABELA
	Chave Primária (PK)	id (PK)
	Chave Estrangeira (FK)	usuario_id (FK)

Categoria	Elemento	Cor / Exemplo
	PK e FK Combinadas	aluno_id (PK, FK)
Tipos de Dados	VARCHAR	VARCHAR
	BOOLEAN	BOOLEAN
	TIMESTAMP	TIMESTAMP
	INT / BIGINT	INT / BIGSERIAL
	TEXT	TEXT
	DATE	DATE
	SERIAL	SERIAL
Obrigatoriedade / Constraints	NOT NULL	NOT NULL
	UNIQUE	UNIQUE
	DEFAULT / CHECK / REFERENCES	DEFAULT / REFERENCES

Código DDL Formatado

```
CREATE TABLE public.usuarios (
  id SERIAL PRIMARY KEY,
  nome VARCHAR(150) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  senha_hash VARCHAR(255) NOT NULL,
  tipo VARCHAR(50) NOT NULL,
```

```
data_nascimento DATE,  
ativo BOOLEAN DEFAULT TRUE,  
criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE public.turmas (  
  id SERIAL PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  ano_letivo INT NOT NULL,  
  descricao TEXT,  
  criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE public.matriculas_turma (  
  aluno_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE  
  CASCADE,  
  turma_id INT NOT NULL REFERENCES public.turmas(id) ON DELETE CASCADE,  
  data_matricula TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  status VARCHAR(50) DEFAULT 'ativo',  
  PRIMARY KEY (aluno_id, turma_id)  
);
```

```
CREATE TABLE public.notificacoes_usuario (  
  id SERIAL PRIMARY KEY,  
  usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE  
  CASCADE,  
  titulo VARCHAR(150) NOT NULL,  
  mensagem TEXT NOT NULL,  
  lida BOOLEAN DEFAULT FALSE,  
  criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);
```

```
CREATE TABLE public.perfis_acessibilidade (  
    id SERIAL PRIMARY KEY,  
    usuario_id INT UNIQUE NOT NULL REFERENCES public.usuarios(id) ON  
DELETE CASCADE,  
    alto_contraste BOOLEAN DEFAULT FALSE,  
    tamanho_fonte VARCHAR(20) DEFAULT 'padrao',  
    ativar_vlibras BOOLEAN DEFAULT FALSE,  
    leitor_tela_nativo BOOLEAN DEFAULT FALSE,  
    reduzir_animacoes BOOLEAN DEFAULT FALSE,  
    fonte_dislexia BOOLEAN DEFAULT FALSE,  
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE public.necessidades_especiais (  
    id SERIAL PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL UNIQUE,  
    descricao TEXT,  
    codigo_cid VARCHAR(20)  
);
```

```
CREATE TABLE public.feedback_acessibilidade (  
    id SERIAL PRIMARY KEY,  
    usuario_id INT REFERENCES public.usuarios(id) ON DELETE SET NULL,  
    pagina_url VARCHAR(255) NOT NULL,  
    comentario TEXT NOT NULL,  
    nota INT CHECK (nota BETWEEN 1 AND 5),  
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE public.usuario_necessidades_especiais (  
    usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE  
    CASCADE,  
    necessidade_id INT NOT NULL REFERENCES  
    public.necessidades_especiais(id) ON DELETE CASCADE,  
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (usuario_id, necessidade_id)  
);
```

```
CREATE TABLE public.tecnologias_assistivas_usuario (  
    id SERIAL PRIMARY KEY,  
    usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE  
    CASCADE,  
    nome_tecnologia VARCHAR(100) NOT NULL,  
    versao VARCHAR(20),  
    atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE public.materias (  
    id SERIAL PRIMARY KEY,  
    nome VARCHAR(150) NOT NULL,  
    descricao TEXT,  
    ordem INT DEFAULT 1  
);
```

```
CREATE TABLE public.modulos (  
    id SERIAL PRIMARY KEY,
```

```
    materia_id INT NOT NULL REFERENCES public.materias(id)
ON DELETE CASCADE,
    titulo VARCHAR(150) NOT NULL,
    descricao TEXT,
    ordem INT DEFAULT 1
);
```

```
CREATE TABLE public.licoes (
    id SERIAL PRIMARY KEY,
    modulo_id INT NOT NULL REFERENCES public.modulos(id) ON
DELETE CASCADE,
    titulo VARCHAR(150) NOT NULL,
    ordem INT DEFAULT 1,
    duracao_estimada_minutos INT DEFAULT 15
);
```

```
CREATE TABLE public.conteudos (
    id SERIAL PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    tipo VARCHAR(50) NOT NULL,
    corpo_texto TEXT,
    url_midia VARCHAR(255),
    transcricao_audio TEXT
);
```

```
CREATE TABLE public.conteudos_licao (
    licao_id INT NOT NULL REFERENCES public.licoes(id) ON
DELETE CASCADE,
    conteudo_id INT NOT NULL REFERENCES
public.conteudos(id) ON DELETE CASCADE,
    ordem INT DEFAULT 1,
    PRIMARY KEY (licao_id, conteudo_id)
```

```
);
```

```
CREATE TABLE public.quizzes (  
    id SERIAL PRIMARY KEY,  
    licao_id INT NOT NULL REFERENCES public.licoes(id) ON  
DELETE CASCADE,  
    titulo VARCHAR(150) NOT NULL,  
    descricao TEXT  
);
```

```
CREATE TABLE public.questoes (  
    id SERIAL PRIMARY KEY,  
    quiz_id INT NOT NULL REFERENCES public.quizzes(id) ON  
DELETE CASCADE,  
    enunciado TEXT NOT NULL,  
    dica_acessivel TEXT  
);
```

```
CREATE TABLE public.alternativas (  
    id SERIAL PRIMARY KEY,  
    questao_id INT NOT NULL REFERENCES public.questoes(id) ON  
DELETE CASCADE,  
    texto_alternativa TEXT NOT NULL,  
    is_correta BOOLEAN DEFAULT FALSE  
);
```

```

CREATE TABLE public.respostas_usuario (
    id SERIAL PRIMARY KEY,
    usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON
DELETE CASCADE,
    questao_id INT NOT NULL REFERENCES public.questoes(id) ON
DELETE CASCADE,
    alternativa_id INT REFERENCES public.alternativas(id),
    resposta_dissertativa TEXT,
    acertou BOOLEAN,
    respondido_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

CREATE TABLE public.progresso_licoes (
    usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON
DELETE CASCADE,
    licao_id INT NOT NULL REFERENCES public.licoes(id) ON
DELETE CASCADE,
    concluido BOOLEAN DEFAULT FALSE,
    data_conclusao TIMESTAMP,
    PRIMARY KEY (usuario_id, licao_id)
);

```

```

CREATE TABLE public.sesoes_estudo (
    id SERIAL PRIMARY KEY,
    usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE,
    inicio TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    fim TIMESTAMP,
    duracao_segundos INT
);

```

```

CREATE TABLE public.conquistas (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    descricao TEXT NOT NULL,
    icone_url VARCHAR(255),
    pontos INT DEFAULT 10
);

```

```

CREATE TABLE public.conquistas_usuario (
    usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE,
    conquista_id INT NOT NULL REFERENCES public.conquistas(id) ON DELETE

```

```

CASCADE,
    desbloqueado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (usuario_id, conquista_id)
);

CREATE TABLE public.recompensas_conquistas (
    id SERIAL PRIMARY KEY,
    conquista_id INT NOT NULL REFERENCES public.conquistas(id) ON DELETE
CASCADE,
    tipo_recompensa VARCHAR(50) NOT NULL,
    valor_recompensa VARCHAR(255) NOT NULL
);

CREATE TABLE public.progresso_gamificacao (
    usuario_id INT PRIMARY KEY REFERENCES public.usuarios(id) ON DELETE
CASCADE,
    pontos_xp INT DEFAULT 0,
    nivel INT DEFAULT 1,
    dias_ofensiva INT DEFAULT 0,
    ultima_atividade DATE DEFAULT CURRENT_DATE
);

CREATE TABLE public.decks_flashcards_usuario (
    id SERIAL PRIMARY KEY,
    usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE,
    titulo VARCHAR(150) NOT NULL,
    descricao TEXT,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE public.cards_memorizacao_usuario (
    id SERIAL PRIMARY KEY,
    deck_id INT NOT NULL REFERENCES public.decks_flashcards_usuario(id) ON
DELETE CASCADE,
    frente TEXT NOT NULL,
    verso TEXT NOT NULL,
    proxima_revisao DATE DEFAULT CURRENT_DATE
);

CREATE TABLE public.logs_aprendizado_ia (
    id SERIAL PRIMARY KEY,
    usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE,
    acao_recomendada VARCHAR(150) NOT NULL,
    justificativa_ia TEXT,
    gerado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE public.mapeamento_revisao_ia (
    id SERIAL PRIMARY KEY,
    usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE,
    licao_id INT NOT NULL REFERENCES public.licoes(id) ON DELETE CASCADE,

```

```

    prioridade INT DEFAULT 1,
    data_sugerida_revisao DATE NOT NULL
);

CREATE TABLE public.logs_acessibilidade_ia (
    id BIGSERIAL PRIMARY KEY,
    usuario_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE,
    recurso_acessibilidade_utilizado VARCHAR(100) NOT NULL,
    acao_usuario VARCHAR(50) NOT NULL,
    tempo_uso_segundos INT,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE public.forum_topicos (
    id SERIAL PRIMARY KEY,
    autor_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE,
    titulo VARCHAR(200) NOT NULL,
    conteudo TEXT NOT NULL,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE public.forum_respostas (
    id SERIAL PRIMARY KEY,
    topico_id INT NOT NULL REFERENCES public.forum_topicos(id) ON DELETE
CASCADE,
    autor_id INT NOT NULL REFERENCES public.usuarios(id) ON DELETE CASCADE,
    resposta TEXT NOT NULL,
    criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

aula 4

Aula 4 - Primeira execução do banco

O que vamos aprender	Execução, mensagem de erro, hipótese e evidência.
O que entregar	Registro de erros da primeira execução.

CONCEITO

Executar um script significa pedir ao SGBD que interprete os comandos. Uma mensagem de erro é uma pista: registre-a antes de tentar “consertar tudo”.

Escolha o caminho da aula

NO LABORATÓRIO Em um PostgreSQL disponível, execute o schema_v1.sql em um ambiente vazio. Para cada falha, copie a mensagem, identifique o comando e formule uma hipótese.	FORA DO LABORATÓRIO - DESPLUGADA Faça a “execução humana” do script. Um estudante lê cada comando; outro representa o banco e cria cartões de tabela. Quando aparecer uma FK, só aceite se o cartão da tabela referenciada já existir. Registre falhas lógicas/sintáticas suspeitas para testar depois.
--	---

Erro / suspeita	Comando	Nossa hipótese	Próxima tentativa
IA COMO EXPLICADORA Prompt: “Recebi esta mensagem de erro no PostgreSQL: [erro]. Explique em linguagem simples o que significa. Não reescreva todo o banco; diga primeiro onde devo procurar.”			

SAIBA MAIS - [Supabase SQL Editor](#)

O SQL Editor também pode ser usado para executar consultas PostgreSQL diretamente no navegador.

O passo a passo da atividade

1. **Prepare um ambiente limpo**
 - Garanta que o banco de dados PostgreSQL esteja zerado (sem tabelas criadas anteriormente) para evitar falsos erros de tabelas já existentes.
2. **Execute o script `schema_v1.sql`**
 - Rode o arquivo completo no terminal, via pgAdmin ou no editor SQL (como o Supabase SQL Editor).
3. **Mapeie e registre os erros encontrados**
 - Ao encontrar um erro, pare a execução imediatamente e analise a mensagem retornada pelo PostgreSQL.
 - Não saia alterando o código sem antes registrar o problema.
4. **Preencha a Tabela de Registro de Erros**
 - Preencha os detalhes da falha no formato solicitado:
 - **Erro / suspeita:** Cole a mensagem de erro exata retornada pelo PostgreSQL (ex.: `ERROR: relation "cliente" does not exist`).
 - **Comando:** Identifique a linha ou bloco SQL que causou a falha (ex.: `CONSTRAINT fk_cliente FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)`).
 - **Nossa hipótese:** Explique o motivo provável da falha (ex.: *A tabela `cliente` está sendo referenciada antes de ser criada no script*).
 - **Próxima tentativa:** Defina a ação corretiva que será testada na próxima aula (ex.: *Mover a instrução de criação da tabela `cliente` para antes da tabela `pedido`*).

Usando a IA como Explicadora

Se você receber uma mensagem de erro complexa ou confusa do PostgreSQL:

- Utilize o prompt indicado: *"Recebi esta mensagem de erro no PostgreSQL: [cole a mensagem de erro aqui]. Explique em linguagem simples o que significa. Não reescreva todo o banco; diga primeiro onde devo procurar."*
- Use a resposta da IA para entender a causa raiz do problema e validar a sua hipótese antes de tentar corrigir a linha de código.

Entregável da Aula 4

A entrega final desta etapa é o **Registro de Erros da primeira execução**, contendo todas as mensagens capturadas, os comandos associados, as hipóteses elaboradas e os testes planejados.

do	Nossa Hipótese	Próxima Tentativa
do ZoJL7E2zHn...	O comando \restrict é uma instrução do utilitário pg_dump do terminal e não uma sintaxe válida de SQL para o PostgreSQL.	Apagar as linhas \res
CREATE TABLE Atividade (...)	A tabela Atividade tentou criar uma FK para Turma antes de Turma ter sido criada.	Reordenar a ordem c Turma antes de Ativi
CREATE TABLE (...) DEFAULT nextval(...)	O script tentou referenciar uma SEQUENCE para auto-incremento antes de criá-la explicitamente.	Substituir a SEQUEN DEFAULT AS IDENT
public.usuarios (...) FROM stdin;	Blocos de importação de dados gerados pelo dump que não são interpretados pelo SQL Editor.	Apagar todas as instr DDL de CREATE TAB
...	O script continha comandos de exportação do terminal (pg_dump) e sintaxe legada de SEQUENCE que o SQL Editor rejeita.	O script foi refatorado para IDENTITY e reo

aula 5

Aula 5 - Corrigir até ser reproduzível

O que vamos aprender	Ciclo executar → compreender → corrigir → recriar → executar.
O que entregar	schema_v2.sql capaz de reconstruir o banco sem correções manuais ocultas.

CONCEITO

Reproduzibilidade significa que outra pessoa consegue recriar a mesma estrutura a partir do arquivo oficial. “Funcionou porque ajustamos manualmente depois” não é reproduzível.

Escolha o caminho da aula

NO LABORATÓRIO Corrija uma falha por vez no arquivo fonte, recrie o ambiente e execute novamente. Repita até o script completo funcionar do início ao fim.	FORA DO LABORATÓRIO - DESPLUGADA Use os cartões/tabelas da aula anterior. Para cada erro registrado, escreva em um novo cartão: causa provável → correção → como testar. Reordene fisicamente comandos quando a dependência exigir.
--	---

1. Escolha um erro.
2. Explique a causa com suas palavras.
3. Faça a menor correção possível.
4. Execute/revise novamente desde o início.
5. Registre se a hipótese foi confirmada.
6. Salve schema_v2.sql somente quando a sequência estiver consistente.

IA PARA CONFIRMAR APRENDIZAGEM

Prompt: “Explique por que esta alteração resolveu o erro. Quero confirmar se entendi e não apenas copiar a solução.”

https://drive.google.com/file/d/1Fbt4-5MtlZ8XugCdI3pCtx7PX_o00CZO/view?usp=drive_link

- **Erro Identificado:** O arquivo `schema_v1.sql` apresentou falhas de execução por conter comandos de terminal do utilitário `pg_dump` (como `\restrict`) e por tentar criar tabelas filhas antes das tabelas principais.
-

- **Causa do Problema:** O arquivo original era uma exportação bruta do banco de dados. Por isso, a ordem de criação dos comandos não respeitava a dependência das Chaves Estrangeiras (FKs), e o editor SQL não reconhecia os comandos iniciados com barra invertida (\).
-
- **Correção Aplicada:**
- - Removemos todas as linhas de terminal que começavam com \.
 -
 - Reordenamos o código para criar primeiro as tabelas independentes (como `usuarios` e `turmas`) e, depois, as tabelas que dependem delas.
 -
 - Atualizamos a sintaxe de auto-incremento dos IDs para o padrão moderno do PostgreSQL (`GENERATED BY DEFAULT AS IDENTITY`).
 -
- **Teste e Validação:** Executamos o script ajustado em um ambiente limpo no SQL Editor do Supabase.
-
- **Resultado e Confirmação:** O PostgreSQL executou a instrução sem falhas e retornou a mensagem `Success. No rows returned`. Isso confirmou nossa hipótese e validou que o novo arquivo (`schema_v2.sql`) está 100% reproduzível.

aula 6

Aula 6 - CRUD: provar que os dados podem ser usados

O que vamos aprender	INSERT, SELECT, UPDATE e DELETE.
O que entregar	testes_crud.sql + resultados esperados/obtidos.

CONCEITO

CRUD representa Create, Read, Update e Delete. Em SQL, normalmente usamos INSERT, SELECT, UPDATE e DELETE. Antes de executar um teste, declare o resultado esperado.

```
INSERT INTO categoria (nome) VALUES ('Bebidas');  
SELECT * FROM categoria;  
UPDATE categoria SET nome='Bebidas sem álcool' WHERE id_categoria=1;  
DELETE FROM categoria WHERE id_categoria=1;
```

Escolha o caminho da aula

NO LABORATÓRIO Escolha uma tabela simples e uma tabela relacionada. Crie dados fictícios e execute as quatro operações. Compare resultado esperado e obtido.	FORA DO LABORATÓRIO - DESPLUGADA Simule o banco com fichas: cada ficha é uma linha. INSERT adiciona ficha; SELECT localiza; UPDATE altera; DELETE remove. Um colega faz o papel das constraints e pode impedir operações inválidas.
--	---

Operação	Comando/simulação	Esperado	Obtido
INSERT			
SELECT			
UPDATE			
DELETE			

IA PARA DADOS DE TESTE

Prompt: "Crie dados fictícios coerentes para testar estas tabelas. Não altere minha estrutura. Gere apenas exemplos de INSERT e explique a ordem necessária por causa das FKs."

SAIBA MAIS - [PostgreSQL - Inserindo dados](#)

Referência oficial para INSERT e manipulação de dados.

Operação	Comando Executado	Resultado Esperado	Resultado Obtido / Evidência
INSERT	<pre>INSERT INTO usuarios ... INSERT INTO perfis_acessibilidade ...</pre>	Inserção de 1 linha de usuário e 1 linha vinculada de acessibilidade.	<pre>INSERT 0 1</pre> em ambos os comandos no Supabase. Registro gravado sem erros de integridade.
SELECT	<pre>SELECT u.id, u.nome, u.email, p.alto_contraste, p.tamanho_fonte FROM usuarios u JOIN perfis_acessibilidade p ON u.id = p.usuario_id ...</pre>	Retorno das colunas solicitadas combinando os dados do usuário e de suas preferências de acessibilidade.	Confirmado via print: O SGBD retornou a linha com <pre>id = 1, nome = Ana Silva, email = ana.silva@email.com, alto_contraste = true e tamanho_fonte = grande.</pre>

UPDATE	<pre>UPDATE perfis_acessibilidade SET tamanho_fonte = 'extra_grande', fonte_dislexia = true WHERE usuario_id = 1;</pre>	Alteração das preferências de acessibilidade para 'extra_grande' e ativação da fonte para dislexia.	UPDATE 1. Os valores da linha correspondente foram atualizados com sucesso.
DELETE	<pre>DELETE FROM usuarios WHERE id = 1;</pre>	Exclusão do usuário ID 1 e remoção automática em cascata da tabela vinculada <code>perfis_acessibilidade</code>.	DELETE 1. O registro foi removido e a restrição ON DELETE CASCADE apagou o perfil associado.

aula 7

Aula 7 - Testes negativos e integridade

O que vamos aprender	Testar também aquilo que o banco deve rejeitar.
O que entregar	Matriz de testes de integridade.

CONCEITO

Um teste positivo verifica o que deve funcionar. Um teste negativo tenta violar uma regra de propósito. Se um email é UNIQUE, repetir o mesmo email deve falhar. Se uma FK exige um registro pai, apontar para um ID inexistente deve falhar.

Escolha o caminho da aula

NO LABORATÓRIO Crie testes que violem NOT NULL, UNIQUE, PK e FK existentes no seu projeto. Antes de rodar, escreva o resultado esperado.	FORA DO LABORATÓRIO - DESPLUGADA Monte uma "portaria do banco": cartões representam registros e estudantes representam constraints. Cada cartão de entrada é aceito ou recusado, mas a equipe deve citar a regra que justifica a decisão.
--	---

Teste	Regra	Esperado	Obtido	Passou?
IA COMO GERADORA DE CENÁRIOS Prompt: "Para estas tabelas PostgreSQL, proponha testes negativos para verificar as constraints que realmente existem no meu script. Não invente regras novas."				

SAIBA MAIS - [PostgreSQL - Constraints](#)

Use a documentação para confirmar o comportamento das restrições.

Teste	Regra Violada	Resultado Esperado	Resultado Obtido / Evidência dos Prints	Passo u?
1. Inserção sem nome	NOT NULL (nome)	Rejeitar usuário sem o campo obrigatório nome.	ERROR: 23502: null value in column "nome" of relation "usuarios" violates not-null constraint	SIM
2. Email duplicado	UNIQUE (email)	Rejeitar tentativa de cadastrar o mesmo e-mail duas vezes.	ERROR: 23505: duplicate key value violates unique constraint "usuarios_email_key"	SIM
3. Nota fora do limite	CHECK (nota >= 1 AND nota <= 5)	Bloquear o envio de nota igual a 6 na tabela de feedback.	ERROR: 23514: new row for relation "feedback_acessibilidade" violates check constraint "feedback_acessibilidade_nota_check"	SIM

4. Associação inválida	FOREIGN KEY (usuario_id)	Bloquear a criação de perfil para o usuario_id = 9999 (inexistente).	ERROR: 23503: insert or update on table "perfis_acessibilidade" violates foreign key constraint "perfis_acessibilidade_usuario_id_fkey"	SIM
----------------------------------	------------------------------------	--	--	------------

- **Testes Positivos x Negativos:** Enquanto a Aula 6 provou que o sistema funciona com dados corretos, a Aula 7 provou que o banco bloqueia dados inválidos na própria camada de dados.
- **Integridade Relacional:** Nenhuma regra precisou ser validada via código de aplicação (backend); o próprio SGBD garantiu a segurança da informação através das *constraints*.

Prints resposta do banco de dados

Results

Chart

Success. No rows returned

Failed to run sql query: ERROR: 23514: new row for relation "feedback_acessibilidade" violates check constraint "feedback_acessibilidade_nota_check"
DETAIL: Failing row contains (1, null, /home, Muito bom, 6, 2026-08-19 14:21:37.508836).

Failed to run sql query: ERROR: 23503: insert or update on table "perfis_acessibilidade" violates foreign key constraint "perfis_acessibilidade_usuario_id_fkey"
DETAIL: Key (usuario_id)=(9999) is not present in table "usuarios".

Failed to run sql query: ERROR: 23505: duplicate key value violates unique constraint "usuarios_email_key"
DETAIL: Key (email)=(ana.silva@email.com) already exists.

aula 8

Aula 8 - Do banco local ao banco remoto

O que vamos aprender	Servidor, credenciais, SQL Editor, Table Editor e segurança.
O que entregar	Plano de implantação + projeto Supabase preparado (quando houver laboratório).

CONCEITO

No Supabase, cada projeto recebe um banco PostgreSQL completo. O Dashboard possui SQL Editor para executar SQL e Table Editor para visualizar/editar tabelas e linhas. O banco remoto poderá ser acessado pelo backend em outra etapa.

Escolha o caminho da aula

NO LABORATÓRIO Acesse o Supabase, crie/abra o projeto da equipe, reconheça Database, SQL Editor, Table Editor e Connect. Guarde a senha com segurança e NÃO a coloque no GitHub.	FORA DO LABORATÓRIO - DESPLUGADA Desenhe a arquitetura em uma folha: COMPUTADOR/APP → REDE → SERVIDOR → POSTGRESQL. Crie cartões “host”, “porta”, “usuário”, “senha” e “banco”. Classifique quais informações podem aparecer em documentação e quais são segredo.
--	---

1. Identifique onde o banco ficará hospedado.
2. Localize SQL Editor e Table Editor.
3. Identifique onde ficam as opções de conexão (Connect).
4. Defina quem guardará a credencial da equipe.
5. Revise o repositório para garantir que não há senhas.

SEGURANÇA

Nunca coloque senha, token ou connection string completa com segredo em repositório público, print compartilhado ou prompt de IA.

SAIBA MAIS - [Supabase - Database](#)

Visão geral oficial: PostgreSQL, Table Editor, SQL Editor e formas de acesso.

Relatório de Implantação e Segurança — Aula 8

Data: Agosto de 2026

1. Onde Nosso Banco de Dados Funciona

- **Onde fica guardado:** Usamos o Supabase na nuvem (PostgreSQL) para acessar o banco de dados de qualquer lugar.
- **SQL Editor:** Lugar onde rodamos os comandos para criar e testar nossas tabelas ([schema_v2.sql](#)).
- **Table Editor:** Tela parecida com uma planilha onde conseguimos ver as tabelas e os dados salvos.

2. O Que Pode e O Que NÃO Pode Ser Mostrado

O que é?	Pode mostrar no GitHub?	Onde guardamos?
Link do Banco / Host	Sim <pre>postgresql://postgres:[senha do banco]@db.mvgbmwvaoedcjostgijv.supabase.co:5432/postgres</pre>	No relatório do grupo
Chave Pública (anon)	Sim	No código do site e no arquivo .env.example
Senha do Banco e Chave Secreta	Não	Apenas no nosso Google Docs privado

3. Como Protegemos Nossas Senhas

- **Google Docs:** Salvamos nossas senhas em um documento do grupo para não perder as informações ao reiniciar os computadores do laboratório.
-
- **Arquivo [.gitignore](#):** Criamos esse arquivo para proibir o Git de enviar o arquivo [.env](#) com nossas senhas reais para o GitHub.
-

- **Arquivo `.env.example`:** Criamos um modelo no GitHub sem senhas reais para mostrar como o projeto deve ser configurado.
-

4. Conclusão

Nosso projeto no GitHub tem apenas os códigos das tabelas e os testes. Nenhuma senha real foi enviada para a internet e o nosso banco de dados no Supabase está seguro e funcionando de modo íntegro e seguro.

responsável pelas credenciais

Mariana Cuartas Carmona n°26 1ºB DS

email: mariana.carmona@escola.pr.gov.br

aula 9

Aula 9 - Implantação no Supabase

O que vamos aprender

Implantar é criar no ambiente remoto a estrutura já validada.

O que entregar

Banco remoto criado a partir do schema.sql.

CONCEITO

O arquivo schema.sql continua sendo a fonte oficial. Se houver erro estrutural, corrija o arquivo e depois reaplique; não deixe uma “versão secreta” que só existe no Dashboard.

Escolha o caminho da aula

NO LABORATÓRIO

No Supabase: abra SQL Editor → New Query → cole o schema.sql validado → Run. Depois confira as tabelas no Table Editor. Registre qualquer erro.

FORA DO LABORATÓRIO - DESPLUGADA

Faça uma implantação simulada: uma folha representa o servidor remoto vazio. Execute o schema em cartões, criando tabelas na ordem. Um “auditor remoto” só aceita o que está no arquivo oficial.

1. Confirme que está usando a versão final do schema.
2. Abra o SQL Editor.
3. Crie uma nova consulta.
4. Execute o schema.
5. Confira tabelas e relacionamentos.
6. Registre evidências da implantação sem expor credenciais.

PERGUNTA DE CONTROLE

Se o banco remoto fosse apagado agora, sua equipe conseguiria reconstruí-lo usando somente os arquivos versionados? A resposta precisa ser SIM.

SAIBA MAIS - [Supabase - SQL Editor](#)

Interface do Supabase para escrever e executar SQL no navegador.

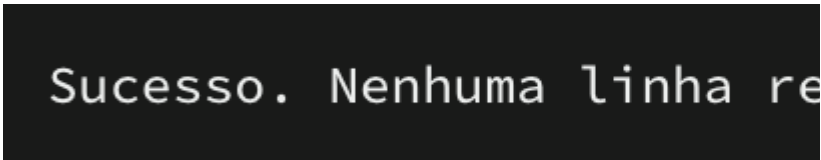
O passo a passo da atividade

1. Confirme a versão final do script

- Garanta que está utilizando o arquivo **schema_v2.sql** (aquele totalmente corrigido, testado e reproduzível obtido na Aula 5).
- 2. **Execute o script via SQL Editor**
 - No painel do Supabase, acesse **SQL Editor** no menu lateral.
 - Clique em **New Query** (Nova Consulta).
 - Cole todo o conteúdo do seu script **schema_v2.sql** e clique em **Run** (ou pressione **Ctrl + Enter**).
- 3. **Valide a estrutura criada**
 - Acesse o **Table Editor** e verifique se todas as tabelas foram criadas com sucesso.
 - Confirme se as chaves primárias (PK), chaves estrangeiras (FK) e restrições (**NOT NULL**, **UNIQUE**) aparecem corretamente nas definições de cada tabela.
- 4. **Teste a Pergunta de Controle da Equipe**
 - Faça a reflexão: *Se o banco do Supabase fosse totalmente apagado neste instante, conseguiríamos recriá-lo em menos de 2 minutos apenas rodando o nosso arquivo .sql?*
 - Se a resposta for **SIM**, sua implantação foi bem-sucedida e reproduzível.

Registro de Evidências da Implantação

Preencha o checklist de confirmação para entregar junto ao projeto:

Item de Verificação	Status	Evidência de Validação
Versão do Script	☒	schema_v2.sql utilizado na íntegra
Execução no SQL Editor	☒	Mensagem Success. No rows returned exibida no Supabase: 

Tabelas Criadas	☒	Todas as tabelas visíveis no Table Editor: alternativas cartões_memorizacao_u... conquistas conquistas_usuario contados conteudos_licao decks_flashcards_usuario feedback_acessibilidade respostas_do_fórum tópicos_do_fórum piolhos logs_acessibilidade_ia logs_aprendido_ia mapeamento_revisão_ia matérias matrículas_turma módulos Especialidades da semana notificacoes_usuario perfis_acessibilidade progresso_gamificação progresso_licoes questoes questionários recompensas_conquistas respostas_usuario sessoes_estudo turmas usuario_necessidades_e... usuários
Relacionament os (FKs)	☒	Todas as chaves estrangeiras(PKs) estão relacionadas corretar
Segurança das Credenciais	☒	Nenhuma senha exposta em prints ou relatórios

aula 10

Aula 10 - Homologação: local × remoto

O que vamos aprender	Repetir os testes no ambiente implantado e comparar comportamentos.
O que entregar	Relatório de homologação local × remoto.

CONCEITO

Homologar é verificar se o ambiente implantado apresenta o comportamento esperado. Criar as tabelas não basta: os testes precisam confirmar CRUD e integridade.

Escolha o caminho da aula

NO LABORATÓRIO Reexecute testes_crud.sql e os testes negativos no Supabase. Compare os resultados com o ambiente anterior e investigue diferenças.	FORA DO LABORATÓRIO - DESPLUGADA Use duas áreas na sala: LOCAL e REMOTO. Reaplique os mesmos cartões de teste nos dois “ambientes”. Qualquer resultado diferente precisa gerar uma hipótese de investigação.
--	--

Teste	Local	Remoto	Conclusão
IA PARA INVESTIGAÇÃO Prompt: “Este teste teve resultado diferente entre dois ambientes PostgreSQL. Ajude-me a criar hipóteses de investigação sem mudar a regra de negócio nem expor credenciais.”			

Na Aula 10, você realiza o processo de **homologação**. O objetivo é pegar os scripts de testes positivos (CRUD da Aula 6) e de testes negativos (integridade/constraints da Aula 7) e reexecutá-los no **ambiente remoto (Supabase)**, comparando o comportamento do banco remoto com o que foi obtido no **ambiente local**.

A meta desta aula é comprovar que o banco hospedado na nuvem responde **exatamente da mesma forma** que o banco de dados que você testou localmente na sua máquina.

O passo a passo da atividade

1. **Reexecute o script de CRUD (`testes_crud.sql`) no Supabase**
 - Rode no SQL Editor as instruções de **INSERT**, **SELECT**, **UPDATE** e **DELETE**.
 - Verifique se as inserções geram as chaves primárias corretamente e se as consultas retornam exatamente os mesmos dados que retornavam no ambiente local.
2. **Reexecute os Testes Negativos no Supabase**
 - Rode no SQL Editor do Supabase os comandos que foram desenhados para falhar (tentativa de e-mail duplicado em campo **UNIQUE**, inserção de **NULL** em campo obrigatório, referência a ID inexistente em **FOREIGN KEY**).
 - Verifique se o PostgreSQL do Supabase bloqueia as operações e se as mensagens de erro de restrição (*constraints*) são idênticas às recebidas localmente.
3. **Compare os comportamentos**
 - Qualquer comportamento diferente entre o ambiente local e o remoto deve ser investigado (ex.: divergência de fusos horários/timezone em campos **TIMESTAMP**, diferenças de versão do PostgreSQL ou extensões não habilitadas).
4. **Preencha a Tabela de Relatório de Homologação**

Teste	Local(PostgreSQL)	Re
CREATE (INSERT) -- Insere um usuário e, em seguida, um perfil	<pre>educainclui=# INSERT INTO public.usuarios (nome, email, senha_hash, tipo, data_nascimento) educainclui-# VALUES ('Ana Silva', 'ana.silva@email.com', 'hash123456', 'aluno', '2005-04-12'); educainclui-# INSERT 0 1 educainclui=# educainclui=# INSERT INTO public.perfis_acessibilidade (usuario_id, alto_contraste, tamanho_fonte) educainclui-# VALUES (1, true, 'grande'); educainclui-# INSERT 0 1</pre>	

de acessibilidade vinculado a ele.		
READ (SELECT) -- Consulta o usuário criado e faz um JOIN com a tabela de acessibilidade.	<pre> educainclui=# SELECT educainclui-# u.id, educainclui-# u.nome, educainclui-# u.email, educainclui-# p.alto_contraste, educainclui-# p.tamanho_fonte educainclui-# FROM public.usuarios educainclui-# JOIN public.perfis_ac educainclui-# WHERE u.email = 'ana. id nome email -----+-----+----- 1 Ana Silva ana.silva@email.c (1 row) </pre>	
UPDATE -- Atualiza as preferências de acessibilidade do usuário.	<pre> educainclui=# UPDATE public.perfis_ educainclui-# SET tamanho_fonte = ' educainclui-# WHERE usuario_id = 1; UPDATE 1 </pre>	
DELETE -- Remove o usuário (graças ao ON DELETE CASCADE, o perfil de acessibilidade também é excluído).	<pre> educainclui=# DELETE FROM public.us educainclui-# WHERE id = 1; DELETE 1 </pre>	

Teste NOT NULL (Tentar criar usuário sem nome)	<pre>educainclui=# INSERT INTO public.usu educainclui=# VALUES ('teste.sem.nom ERROR: null value in column "nome" DETAIL: Failing row contains (2, nu</pre>
Teste UNIQUE (Tentar cadastrar email já existente)	<pre>educainclui=# INSERT INTO public.usu educainclui=# VALUES ('Outra Ana', ERROR: duplicate key value violate DETAIL: Key (email)=(ana.silva@ema</pre>
Teste CHECK (Tentar dar nota 6 no feedback_acessibilidade)	<pre>educainclui=# INSERT INTO public.fe educainclui=# VALUES ('/home', 'Mui ERROR: new row for relation "feedb DETAIL: Failing row contains (1, n</pre>
Teste Chave Estrangeira / FK (Tentar vincular perfil a usuario_id inexistente)	<pre>educainclui=# INSERT INTO public.pe educainclui=# VALUES (9999, true); ERROR: insert or update on table " DETAIL: Key (usuario_id)=(9999) is</pre>

CONCLUSÃO FINAL:

Os códigos funcionam em ambos os ambientes, tanto no postgres, como no supabase

aula 11

Aula 11 - Consolidar: tudo precisa dizer a mesma coisa

O que vamos aprender	Sincronizar Dicionário, DER, schema e banco implantado.
O que entregar	Versões finais dos artefatos + checklist de auditoria.

CONCEITO

Documentação desatualizada também é defeito. Ao final, a estrutura descrita no Dicionário e DER deve corresponder ao SQL que reconstrói o banco implantado.

Escolha o caminho da aula

NO LABORATÓRIO Compare Dicionário, DER, schema.sql e estrutura no Supabase. Faça correções finais nos arquivos oficiais e uma auditoria cruzada com outra equipe.	FORA DO LABORATÓRIO - DESPLUGADA Monte quatro estações: DICIONÁRIO, DER, SQL e BANCO IMPLANTADO (representado por desenho/lista). Grupos rodam pelas estações e marcam divergências em post-its sem editar o trabalho da outra equipe.
---	--

Checklist

- ☐ Toda tabela aparece nos quatro lugares.
- ☐ Nomes de campos estão coerentes.
- ☐ Tipos são compatíveis.
- ☐ PKs e FKs correspondem.
- ☐ Obrigatoriedade/restrições correspondem.
- ☐ Toda alteração feita durante os testes voltou para a documentação.

IA COMO AUDITORA

Prompt: “Faça uma auditoria de consistência entre estes artefatos. Não redesenhe a solução. Liste divergências objetivas em nomes, tipos, obrigatoriedade, PK, FK e regras.”

- **Toda tabela aparece nos quatro lugares?** Sim, todas as tabelas aparecem em ambos os locais(dicionário de dados, DER, estrutura no supabase e schema.sql)
- **Nomes de campos estão coerentes?** Sim, todos os nomes de campo/atributos estão coerentes
- **Tipos são compatíveis?** Sim, os tipo de dados são todos compatíveis
- **PKs e FKs correspondem?** Sim, as PKs e FKs são correspondentes/correspondem
- **Obrigatoriedade/restrições correspondem?** Sim, a obrigatoriedade/restrições são correspondentes/correspondem
- **Toda alteração feita durante os testes voltou para a documentação?** Sim, toda alteração feita durante os testes voltou para a documentação

aula 12

Aula 12 - Versionar, entregar e demonstrar

O que vamos aprender	Entrega profissional combina código, documentação, testes, segurança e explicação.
O que entregar	Repositório final organizado + demonstração + pendências para a próxima Sprint.

CONCEITO

O projeto não termina quando “funciona no meu computador”. A equipe deve permitir que outra pessoa compreenda, reconstrua e teste o banco.

Escolha o caminho da aula

NO LABORATÓRIO Organize /docs e /database, revise segredos, faça commit e demonstre no Supabase: uma tabela, um CRUD e um teste negativo.	FORA DO LABORATÓRIO - DESPLUGADA Realize uma banca técnica com cartões-surpresa: explique uma entidade, sua PK, um relacionamento, simule um INSERT válido e um inválido. Use os artefatos impressos como evidência.
---	--

```
/docs
dicionario_dados
DER
relatorio_validacao
/database
schema.sql
testes_crud.sql
testes_integridade.sql
```

Atividade transversal - Publicação da Sprint no Instagram do projeto

Toda semana de Projeto Integrador, a equipe deverá publicar uma evidência do desenvolvimento realizado no Instagram do próprio projeto. A postagem faz parte da entrega da Sprint.

Nesta semana, a publicação deve contar a transformação do banco projetado em um banco executável, testado e implantado.

1. Escolha uma evidência real da Sprint: DER, trecho do schema.sql, execução de teste, tabela criada no Supabase ou outro registro do trabalho da equipe.
2. Explique, em linguagem acessível, o que a equipe fez nesta semana.
3. Conte pelo menos um problema encontrado durante execução, teste ou implantação e como a equipe investigou/corrigiu.
4. Mostre o avanço obtido: o que o projeto passou a conseguir fazer depois desta Sprint?
5. Revise imagem e texto antes de publicar. Não exponha senha, token, chave, connection string, endereço privado ou qualquer credencial.
6. Publique no Instagram do projeto e registre a evidência/link da postagem no material de entrega da equipe.

Uso da IA na publicação

A IA pode ajudar a transformar o registro técnico da Sprint em uma legenda compreensível. Exemplo de prompt: “Com base neste resumo do que realmente fizemos, ajude nossa equipe a escrever uma legenda curta para o Instagram do projeto. Explique o avanço técnico em linguagem acessível, sem inventar resultados e sem incluir credenciais.”

Importante: a equipe deve conferir se a publicação representa aquilo que foi realmente desenvolvido. IA ajuda a comunicar; não cria uma entrega que não aconteceu.

Registro da publicação	Preenchimento da equipe
Evidência escolhida	
Problema/avanço que será contado	
Legenda revisada pela equipe?	
Link ou identificação da publicação	

Checklist de entrega

- ☐ Dicionário, DER e schema.sql coerentes.
- ☐ Schema reproduzível.
- ☐ Testes CRUD registrados.
- ☐ Testes negativos registrados.
- ☐ Banco remoto homologado.
- ☐ Nenhum segredo versionado.
- ☐ Arquivos organizados no GitHub.
- ☐ Equipe consegue explicar uma entidade, um relacionamento, um CRUD e um teste negativo.
- ☐ Publicação semanal no Instagram do projeto realizada e registrada, sem exposição de credenciais.

IA COMO BANCA

Prompt: “Atue como banca técnica. Faça uma pergunta por vez para verificar se entendemos nosso banco, testes e implantação. Espere nossa tentativa antes de explicar a resposta.”

Guia rápido de SQL para esta Sprint

CREATE TABLE

Cria a estrutura de uma tabela. O DDL deve refletir as decisões do Dicionário e do DER.

```
CREATE TABLE produto (  
  id_produto INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  nome VARCHAR(120) NOT NULL,  
  preco NUMERIC(10,2) NOT NULL  
);
```

INSERT / SELECT / UPDATE / DELETE

São os comandos básicos usados nesta Sprint para verificar CRUD. Sempre use dados fictícios de teste e confira o resultado.

SAIBA MAIS - [PostgreSQL - Tutorial oficial](#)

Percurso introdutório oficial sobre bancos relacionais e SQL.

SAIBA MAIS - [PostgreSQL - UPDATE](#)

Exemplos oficiais de atualização de linhas.

Como usar IA sem terceirizar sua aprendizagem

1. Antes do prompt, escreva sua hipótese.
2. Peça explicação antes de pedir código.
3. Compare a resposta com Dicionário, DER e regras do projeto.
4. Execute ou simule a proposta.
5. Registre a evidência que confirmou ou rejeitou a sugestão.

PROMPT-BASE

“Somos estudantes desenvolvendo um projeto autoral com PostgreSQL. Atue como tutor. Primeiro explique o conceito necessário, depois ajude a localizar a causa, proponha passos de investigação e só então uma possível correção. Termine dizendo como testar se funcionou.”

Papéis da equipe

Papel	Responsabilidade
Operador	Executa comandos quando houver computador.
Auditor	Compara Dicionário, DER, SQL e resultados.
Analista de erro	Registra mensagens, hipóteses e evidências.
Testador	Cria cenários positivos e negativos.
Documentador	Atualiza relatórios e checklists.
Consultor de IA	Formula prompts e valida respostas.
Guardião de segurança	Confere que segredos não sejam expostos.
ROTACIONE Troque os papéis ao longo da Sprint. O estudante que já domina mais tecnologia não deve ser o único a operar o computador.	

Saiba mais - referências oficiais

- [PostgreSQL Tutorial](#)

- [PostgreSQL Constraints](#)
- [PostgreSQL INSERT](#)
- [Supabase Database](#)
- [Supabase SQL Editor](#)

Nesta semana, a publicação deve contar a transformação do banco projetado em um banco executável, testado e implantado.

1. Escolha uma evidência real da Sprint: DER, trecho do schema.sql, execução de teste, tabela criada no Supabase ou outro registro do trabalho da equipe.

2. Explique, em linguagem acessível, o que a equipe fez nesta semana:

- Essa semana a equipe criou arquivos schema.sql;
- Realizou testes com os códigos das tabelas no postgresql e no supabase;
- Tbm realizamos testes de tabelas com CRUD;
- Comparamos os dados das informações das tabelas que estão no DER, Dicionario de dados, schema.sql e as tabelas originais;
- Criamos nosso banco de dados no supabase;
- Organizamos como está classificado cada informação de certa tela do sistema;
- Tentamos varios testes diferentes envolvendo os codigos no supabase, para ver se algo da erro ou não, para assim concluir se está ou não rodando bem os códigos;
- E mais algumas coisas.

3. Conte pelo menos um problema encontrado durante execução, teste ou implantação e como a equipe investigou/corrigiu.

Um dos problemas encontrados durante a execução dos códigos, ocorreu quando fomos inserir a tabela “Atividades”, porém ela deu erro, pois, a tabela “Turma” a quem a chave estrangeira da tabela atividades referenciava, não havia sido criada ainda/vinha depois dela na ordem dos codigos. Para arrumar, só reordenamos a ordem das tabelas, para que a tabela turma e as demais tabelas principais/independentes viessem primeiro na ordem(antes das tabelas dependentes de outras)

4. Mostre o avanço obtido: o que o projeto passou a conseguir fazer depois desta Sprint?

[???] [???] [???] [???] [???] [???] [???] [???] [???]

Importante: a equipe deve conferir se a publicação representa aquilo que foi realmente desenvolvido. IA ajuda a comunicar; não cria uma entrega que não aconteceu.

Registro da publicação	Preenchimento da equipe
Evidência escolhida	
Problema/avanço que será contado	
Legenda revisada pela equipe?	
Link ou identificação da publicação	

Checklist de entrega

- ☒ ~~Dicionário, DER e schema.sql coerentes.~~
- ☒ ~~Schema reproduzível.~~
- ☒ ~~Testes CRUD registrados.~~
- ☒ ~~Testes negativos registrados.~~
- ☒ ~~Banco remoto homologado.~~
- ☐ Nenhum segredo versionado.
- ☒ ~~Arquivos organizados no GitHub.~~
- ☒ ~~Equipe consegue explicar uma entidade, um relacionamento, um CRUD e um teste negativo.~~
- ☐ Publicação semanal no Instagram do projeto realizada e registrada, sem exposição de credenciais.